

CO4- AT2 - Lab Practice

Java Lab Practice – Applets, AWT Components and Layout Managers

Name : M.Chandana

Reg Num : 192472250

Course Code : CSA0925

Course Name : Programming In Java

Question 1 – Applet Life Cycle

Develop a Java Applet to demonstrate the **Applet Life Cycle** methods: init(), start(), paint(), stop(), and destroy().

Display a suitable message in the applet window whenever each life-cycle method is invoked. Observe the sequence of execution by running the applet and interacting with it.

Expected Outcome:

Students should understand the execution sequence and purpose of each Applet life-cycle method.

Question 2 – Generating Graphic Images Using Applet

Develop a Java Applet that uses the **Graphics class** to generate a simple graphical scene.

The applet should draw:

- A house
- Sun
- Trees
- Road
- Clouds

Use methods such as drawLine(), drawRect(), drawOval(), fillRect(), fillOval(), and appropriate colors.

Expected Outcome:

Students should demonstrate the use of the Graphics class for drawing shapes and images.

Question 3 – Calculator Application Using Applet Parameters

Develop a **Calculator Applet** that accepts two numbers and an arithmetic operator through **HTML/Applet parameters**.

The applet should perform:

- Addition
- Subtraction
- Multiplication
- Division

Use the `getParameter()` method to retrieve the values passed to the applet.

Expected Outcome:

Students should understand how parameters are passed from an HTML page to an Applet and processed in Java.

Question 4 – Student Profile Using AWT Components

Develop a **Student Profile Application** using Java AWT components such as:

- Label
- TextField
- Checkbox
- CheckboxGroup
- Choice
- TextArea
- Button
- List

The application should accept student details such as:

- Name
- Register Number
- Gender
- Department
- Year
- Skills/Programming Languages

When the **Generate Report** button is clicked, display the entered student information in a List or TextArea.

Expected Outcome:

Students should demonstrate the use of multiple AWT components and event handling.

Question 5 – GridLayout with Event Handling

Develop a Java AWT application using **GridLayout** to create a simple form or calculator interface.

Arrange the components in a grid and implement appropriate **event handling** using ActionListener.

For example, create a calculator with buttons:

```
7 8 9 +  
4 5 6 -  
1 2 3 *  
0 = C /
```

Perform the corresponding operation when a button is clicked.

Expected Outcome:

Students should understand GridLayout and button-based event handling.

Question 6 – GridBagLayout with Event Handling

Develop a Java AWT **Student Registration Form** using GridBagLayout.

The form should contain:

- Name
- Register Number
- Department
- Email
- Gender
- Year
- Submit Button
- Reset Button

Use GridBagConstraints to properly position and align the components.

Implement event handling for the **Submit** and **Reset** buttons.

Expected Outcome:

Students should understand flexible component positioning using GridBagLayout and event handling.

Question 7 – BorderLayout with Event Handling

Develop a Java AWT application using **BorderLayout**.

Divide the window into:

- NORTH – Header
- SOUTH – Buttons
- EAST – Options
- WEST – Menu
- CENTER – Display area

Add buttons in the interface and implement event handling so that clicking different buttons changes the content displayed in the center area.

Expected Outcome:

Students should understand the five regions of BorderLayout and event-driven programming.

Question 8 – CardLayout with Event Handling

Develop a Java AWT application using **CardLayout** to create a multi-page interface.

Create at least three cards:

- **Card 1:** Student Information
- **Card 2:** Academic Information
- **Card 3:** Confirmation/Report

Provide **Next**, **Previous**, and **Submit** buttons to navigate between the cards.

Implement event handling to switch between cards using CardLayout.

Expected Outcome:

Students should understand how CardLayout can be used to create multiple screens within a single window.

Lab Practice Evaluation Focus

Students will be evaluated based on:

1. Correct implementation of the required Java concepts.
2. Proper use of AWT components.
3. Correct implementation of event handling.
4. Appropriate use of layout managers.
5. User-friendly interface design.
6. Successful compilation and execution.

7. Ability to explain the program and output during the lab examination.

Q1.

LifeCycleApplet.java

```
import java.applet.Applet;
import java.awt.Graphics;

public class LifeCycleApplet extends Applet {

    public void init() {
        System.out.println("init() method called");
    }

    public void start() {
        System.out.println("start() method called");
    }

    public void paint(Graphics g) {
        g.drawString("paint() method called", 50, 50);
    }

    public void stop() {
        System.out.println("stop() method called");
    }

    public void destroy() {
        System.out.println("destroy() method called");
    }
}
```

LifeCycleApplet.html

```
<html>
<body>

<applet code="LifeCycleApplet.class" width="500" height="300">
</applet>
```

```
</body>  
</html>
```

OUTPUT :



paint() method called

Applet started.

Q2.

SceneApplet.java

```
import java.applet.Applet;  
import java.awt.Color;  
import java.awt.Graphics;
```

```
public class SceneApplet extends Applet {
```

```
public void paint(Graphics g) {  
  
    // Sky  
    g.setColor(Color.cyan);  
    g.fillRect(0, 0, 700, 450);  
  
    // Sun  
    g.setColor(Color.yellow);  
    g.fillOval(550, 40, 80, 80);  
  
    // Sun outline  
    g.setColor(Color.orange);  
    g.drawOval(550, 40, 80, 80);  
  
    // Clouds  
    g.setColor(Color.white);  
  
    g.fillOval(80, 60, 80, 40);  
    g.fillOval(120, 40, 100, 60);  
    g.fillOval(180, 60, 80, 40);  
  
    g.fillOval(350, 80, 80, 40);  
    g.fillOval(390, 60, 100, 60);  
    g.fillOval(450, 80, 80, 40);  
  
    // Ground  
    g.setColor(Color.green);  
    g.fillRect(0, 280, 700, 170);  
  
    // Road  
    g.setColor(Color.gray);  
    g.fillRect(0, 340, 700, 110);  
  
    // Road divider lines  
    g.setColor(Color.white);  
  
    for (int i = 0; i < 700; i += 100) {  
        g.fillRect(i, 390, 50, 5);  
    }  
}
```

```
// House body
g.setColor(Color.orange);
g.fillRect(230, 180, 220, 160);

// House border
g.setColor(Color.black);
g.drawRect(230, 180, 220, 160);

// Roof
g.setColor(Color.red);

int[] xPoints = {200, 340, 480};
int[] yPoints = {180, 70, 180};

g.fillPolygon(xPoints, yPoints, 3);

// Roof outline
g.setColor(Color.black);
g.drawLine(200, 180, 340, 70);
g.drawLine(340, 70, 480, 180);

// Door
g.setColor(Color.darkGray);
g.fillRect(320, 250, 50, 90);

g.setColor(Color.black);
g.drawRect(320, 250, 50, 90);

// Door knob
g.setColor(Color.yellow);
g.fillOval(355, 290, 8, 8);

// Left window
g.setColor(Color.white);
g.fillRect(255, 220, 50, 50);

g.setColor(Color.black);
g.drawRect(255, 220, 50, 50);
g.drawLine(280, 220, 280, 270);
g.drawLine(255, 245, 305, 245);
```



```

// Right window
g.setColor(Color.white);
g.fillRect(390, 220, 50, 50);

g.setColor(Color.black);
g.drawRect(390, 220, 50, 50);
g.drawLine(415, 220, 415, 270);
g.drawLine(390, 245, 440, 245);

// Tree 1 - trunk
g.setColor(new Color(139, 69, 19));
g.fillRect(80, 210, 35, 100);

// Tree 1 - leaves
g.setColor(Color.green);
g.fillOval(40, 150, 110, 100);
g.fillOval(80, 120, 100, 100);

// Tree 2 - trunk
g.setColor(new Color(139, 69, 19));
g.fillRect(550, 210, 35, 100);

// Tree 2 - leaves
g.setColor(Color.green);
g.fillOval(510, 150, 110, 100);
g.fillOval(550, 120, 100, 100);

// Title
g.setColor(Color.black);
g.drawString("Simple Village Scene", 285, 430);
}
}

```

notepad SceneApplet.html

```

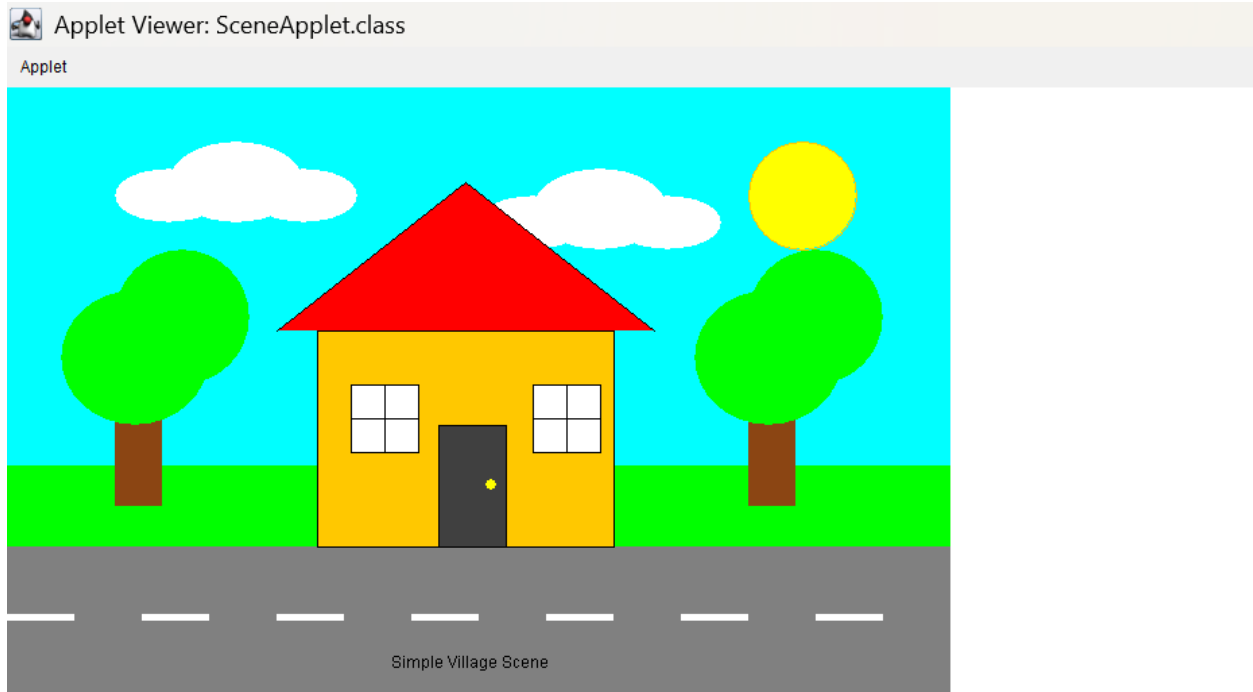
<html>
<body>

<applet code="SceneApplet.class" width="700" height="450">
</applet>

```

```
</body>  
</html>
```

OUTPUT:



Q3.

CalculatorApplet.java

```
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class CalculatorApplet extends Applet {  
  
    public void paint(Graphics g) {  
  
        // Get parameters from HTML  
        String value1 = getParameter("num1");  
        String value2 = getParameter("num2");  
        String operator = getParameter("operator");
```

```

// Convert String values into numbers
double num1 = Double.parseDouble(value1);
double num2 = Double.parseDouble(value2);

double result = 0;

// Perform calculation
if (operator.equals("+")) {
    result = num1 + num2;
}
else if (operator.equals("-")) {
    result = num1 - num2;
}
else if (operator.equals("*")) {
    result = num1 * num2;
}
else if (operator.equals("/")) {

    if (num2 != 0) {
        result = num1 / num2;
    }
    else {
        g.drawString("Cannot divide by zero", 50, 100);
        return;
    }
}

// Display result
g.drawString("CALCULATOR APPLET", 50, 40);
g.drawString("Number 1: " + num1, 50, 70);
g.drawString("Number 2: " + num2, 50, 90);
g.drawString("Operator: " + operator, 50, 110);
g.drawString("Result: " + result, 50, 140);
}
}

```

CalculatorApplet.html

```

<html>
<body>

```

```
<applet code="CalculatorApplet.class"
        width="400"
        height="250">

    <param name="num1" value="20">
    <param name="num2" value="10">
    <param name="operator" value="+">

</applet>

</body>
</html>
```

OUTPUT :



Applet Viewer: CalculatorApplet.class

Applet

CALCULATOR APPLET

Number 1: 20.0

Number 2: 10.0

Operator: +

Result: 30.0

Q4.

StudentProfile.java

```
import java.awt.*;
import java.awt.event.*;

public class StudentProfile extends Frame implements ActionListener {
```

```
Label nameLabel, regLabel, genderLabel;  
Label deptLabel, yearLabel, skillsLabel;
```

```
TextField nameField, regField;
```

```
Checkbox male, female;  
CheckboxGroup genderGroup;
```

```
Choice departmentChoice;  
Choice yearChoice;
```

```
TextArea skillsArea;  
TextArea reportArea;
```

```
List skillsList;
```

```
Button generateButton;  
Button clearButton;
```

```
public StudentProfile() {  
  
    setTitle("Student Profile Application");  
  
    setLayout(new FlowLayout());  
  
    // Name  
    nameLabel = new Label("Name:");  
    nameField = new TextField(25);  
  
    add(nameLabel);  
    add(nameField);  
  
    // Register Number  
    regLabel = new Label("Register Number:");  
    regField = new TextField(25);  
  
    add(regLabel);  
    add(regField);  
  
    // Gender  
    genderLabel = new Label("Gender:");
```

```
genderGroup = new CheckboxGroup();

male = new Checkbox("Male", genderGroup, false);
female = new Checkbox("Female", genderGroup, false);

add(genderLabel);
add(male);
add(female);

// Department
deptLabel = new Label("Department:");

departmentChoice = new Choice();
departmentChoice.add("CSE");
departmentChoice.add("CSE-AI");
departmentChoice.add("ECE");
departmentChoice.add("EEE");
departmentChoice.add("MECH");

add(deptLabel);
add(departmentChoice);

// Year
yearLabel = new Label("Year:");

yearChoice = new Choice();
yearChoice.add("1st Year");
yearChoice.add("2nd Year");
yearChoice.add("3rd Year");
yearChoice.add("4th Year");

add(yearLabel);
add(yearChoice);

// Skills TextArea
skillsLabel = new Label("Skills:");

skillsArea = new TextArea(3, 25);

add(skillsLabel);
```

```
add(skillsArea);

// Skills List
skillsList = new List(4, true);

skillsList.add("Java");
skillsList.add("Python");
skillsList.add("C");
skillsList.add("SQL");
skillsList.add("HTML");
skillsList.add("CSS");

add(new Label("Select Skills:"));
add(skillsList);

// Buttons
generateButton = new Button("Generate Report");
clearButton = new Button("Clear");

add(generateButton);
add(clearButton);

// Report area
reportArea = new TextArea(10, 40);

add(new Label("Student Report:"));
add(reportArea);

// Event handling
generateButton.addActionListener(this);
clearButton.addActionListener(this);

// Window settings
setSize(600, 650);
setVisible(true);

// Close window
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
})
```

```
});  
}
```

```
public void actionPerformed(ActionEvent e) {  
  
    if (e.getSource() == generateButton) {  
  
        String gender = genderGroup.getSelectedCheckbox() != null  
            ? genderGroup.getSelectedCheckbox().getLabel()  
            : "Not Selected";  
  
        String selectedSkills = "";  
  
        String[] selected = skillsList.getSelectedItems();  
  
        for (String skill : selected) {  
            selectedSkills += skill + " ";  
        }  
  
        reportArea.setText(  
            "----- STUDENT PROFILE -----\\n\\n" +  
            "Name: " + nameField.getText() + "\\n" +  
            "Register Number: " + regField.getText() + "\\n" +  
            "Gender: " + gender + "\\n" +  
            "Department: " +  
            departmentChoice.getSelectedItem() + "\\n" +  
            "Year: " +  
            yearChoice.getSelectedItem() + "\\n" +  
            "Skills: " + skillsArea.getText() + "\\n" +  
            "Selected Programming Languages: " +  
            selectedSkills  
        );  
    }  
  
    if (e.getSource() == clearButton) {  
  
        nameField.setText("");  
        regField.setText("");  
        skillsArea.setText("");  
        reportArea.setText("");  
    }  
}
```



```
genderGroup.setSelectedCheckbox(null);
```

```
skillsList.deselect(0);
```

```
skillsList.deselect(1);
```

```
skillsList.deselect(2);
```

```
skillsList.deselect(3);
```

```
skillsList.deselect(4);
```

```
skillsList.deselect(5);
```

```
}
```

```
}
```

```
public static void main(String[] args) {
```

```
    new StudentProfile();
```

```
}
```

```
}
```

OUTPUT :

The screenshot shows the 'Student Profile Application' window. The form is filled with the following data: Name: chandana, Register Number: 192472250, Gender: ☒ Female, Department: CSE, Year: 3rd Year, Skills: java. The 'Selected Skills' list shows Java, Python, C, and SQL. The 'Generate Report' button is visible. A 'Student Report' link is also present. A text box displays the 'STUDENT PROFILE' details: Name: chandana, Register Number: 192472250, Gender: Female, Department: CSE, Year: 3rd Year, Skills: java, and Selected Programming Languages: Java.

The screenshot shows the 'Student Profile Application' window with an empty form. The fields are: Name: , Register Number: , Gender: ☐ Male ☐ Female, Department: CSE, Year: 1st Year, Skills: . The 'Selected Skills' list shows Java, Python, C, and SQL. The 'Generate Report' button is visible. A 'Student Report' link is also present. A large empty text box is at the bottom.

Q5.

GridCalculator.java

```
import java.awt.*;
import java.awt.event.*;

public class GridCalculator extends Frame implements ActionListener {

    TextField display;
    double num1 = 0;
    String operator = "";

    GridCalculator() {

        setTitle("GridLayout Calculator");

        setLayout(new BorderLayout());

        // Display
        display = new TextField();
        display.setEditable(false);
        add(display, BorderLayout.NORTH);

        // Calculator buttons
        Panel panel = new Panel();
        panel.setLayout(new GridLayout(4, 4));

        String[] buttons = {
            "7", "8", "9", "+",
            "4", "5", "6", "-",
            "1", "2", "3", "*",
            "0", "=", "C", "/"
        };

        for (int i = 0; i < buttons.length; i++) {

            Button button = new Button(buttons[i]);

            button.addActionListener(this);
```

```

        panel.add(button);
    }

    add(panel, BorderLayout.CENTER);

    // Close window
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });

    setSize(300, 400);
    setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    String value = e.getActionCommand();

    // Clear
    if (value.equals("C")) {

        display.setText("");
        num1 = 0;
        operator = "";
    }

    // Operator
    else if (value.equals("+") ||
             value.equals("-") ||
             value.equals("*") ||
             value.equals("/")) {

        num1 = Double.parseDouble(display.getText());

        operator = value;

        display.setText("");
    }
}

```

```

// Equal
else if (value.equals("=")) {

    double num2 = Double.parseDouble(display.getText());

    double result = 0;

    if (operator.equals("+"))
        result = num1 + num2;

    else if (operator.equals("-"))
        result = num1 - num2;

    else if (operator.equals("*"))
        result = num1 * num2;

    else if (operator.equals("/")) {

        if (num2 != 0)
            result = num1 / num2;
        else {
            display.setText("Cannot divide by zero");
            return;
        }
    }

    display.setText(String.valueOf(result));
}

// Number
else {

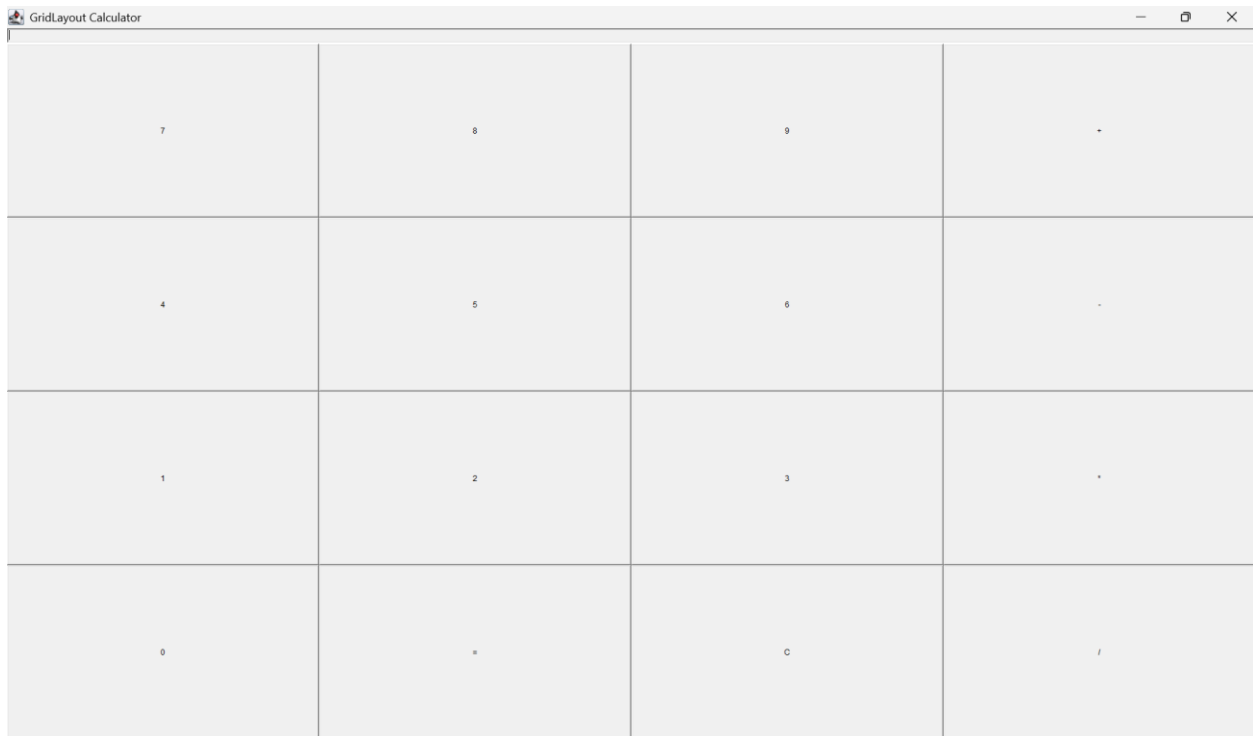
    display.setText(display.getText() + value);
}

}

public static void main(String[] args) {
    new GridCalculator();
}
}

```

OUTPUT:



Q6.

StudentRegistration.java

```
import java.awt.*;
import java.awt.event.*;

public class StudentRegistration extends Frame
    implements ActionListener {

    TextField nameField;
    TextField regField;
    TextField emailField;

    Choice departmentChoice;
    Choice yearChoice;

    Checkbox male;
    Checkbox female;
    CheckboxGroup genderGroup;
```

```
Button submitButton;
```

```
Button resetButton;
```

```
TextArea reportArea;
```

```
StudentRegistration() {
```

```
    setTitle("Student Registration Form");
```

```
    setLayout(new GridBagLayout());
```

```
    GridBagConstraints gbc = new GridBagConstraints();
```

```
    gbc.insets = new Insets(8, 8, 8, 8);
```

```
    gbc.anchor = GridBagConstraints.WEST;
```

```
    // Name
```

```
    addComponent(  
        new Label("Name:"),  
        gbc, 0, 0  
    );
```

```
    nameField = new TextField(25);
```

```
    addComponent(  
        nameField,  
        gbc, 1, 0  
    );
```

```
    // Register Number
```

```
    addComponent(  
        new Label("Register Number:"),  
        gbc, 0, 1  
    );
```

```
    regField = new TextField(25);
```

```
    addComponent(  
        regField,  
        gbc, 1, 1  
    );
```

```
);

// Department
addComponent(
    new Label("Department:"),
    gbc, 0, 2
);

departmentChoice = new Choice();

departmentChoice.add("CSE");
departmentChoice.add("CSE-AI");
departmentChoice.add("ECE");
departmentChoice.add("EEE");
departmentChoice.add("MECH");

addComponent(
    departmentChoice,
    gbc, 1, 2
);

// Email
addComponent(
    new Label("Email:"),
    gbc, 0, 3
);

emailField = new TextField(25);

addComponent(
    emailField,
    gbc, 1, 3
);

// Gender
addComponent(
    new Label("Gender:"),
    gbc, 0, 4
);

genderGroup = new CheckboxGroup();
```

```
male = new Checkbox(  
    "Male",  
    genderGroup,  
    false  
);  
  
female = new Checkbox(  
    "Female",  
    genderGroup,  
    false  
);  
  
addComponent(male, gbc, 1, 4);  
addComponent(female, gbc, 2, 4);  
  
// Year  
addComponent(  
    new Label("Year:"),  
    gbc, 0, 5  
);  
  
yearChoice = new Choice();  
  
yearChoice.add("1st Year");  
yearChoice.add("2nd Year");  
yearChoice.add("3rd Year");  
yearChoice.add("4th Year");  
  
addComponent(  
    yearChoice,  
    gbc, 1, 5  
);  
  
// Submit button  
submitButton = new Button("Submit");  
  
addComponent(  
    submitButton,  
    gbc, 0, 6  
);
```



```

// Reset button
resetButton = new Button("Reset");

addComponent(
    resetButton,
    gbc, 1, 6
);

// Report
addComponent(
    new Label("Report:"),
    gbc, 0, 7
);

reportArea = new TextArea(8, 35);

gbc.fill = GridBagConstraints.BOTH;
gbc.gridwidth = 2;

addComponent(
    reportArea,
    gbc, 1, 7
);

// Event handling
submitButton.addActionListener(this);
resetButton.addActionListener(this);

// Window settings
setSize(600, 550);
setVisible(true);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});
}

```

```

void addComponent(
    Component component,
    GridBagConstraints gbc,
    int x,
    int y) {

    gbc.gridx = x;
    gbc.gridy = y;

    add(component, gbc);
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == submitButton) {

        String gender = "Not Selected";

        if (genderGroup.getSelectedCheckbox() != null) {
            gender =
                genderGroup
                .getSelectedCheckbox()
                .getLabel();
        }

        reportArea.setText(
            "----- STUDENT REGISTRATION -----\\n\\n" +
            "Name: " + nameField.getText() + "\\n" +
            "Register Number: " + regField.getText() + "\\n" +
            "Department: " +
            departmentChoice.getSelectedItemAt() + "\\n" +
            "Email: " + emailField.getText() + "\\n" +
            "Gender: " + gender + "\\n" +
            "Year: " +
            yearChoice.getSelectedItemAt()
        );
    }

    else if (e.getSource() == resetButton) {

        nameField.setText("");
    }
}

```

```

        regField.setText("");
        emailField.setText("");

        reportArea.setText("");

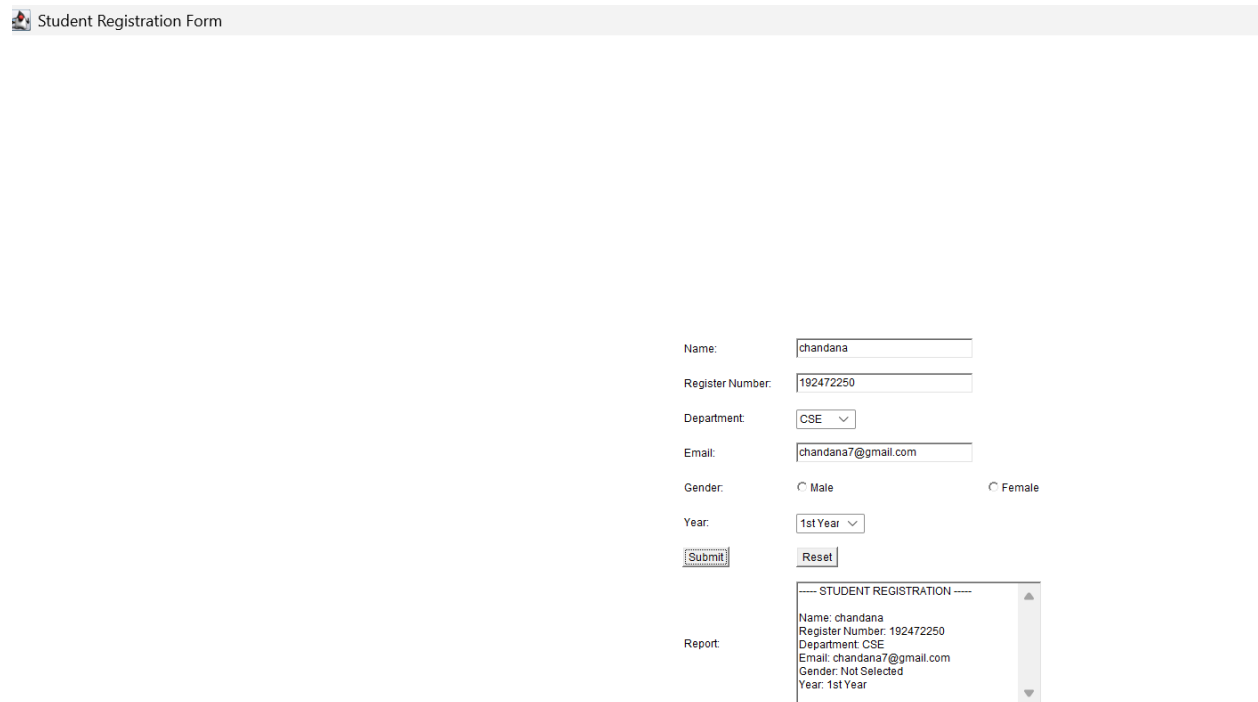
        genderGroup.setSelectedCheckbox(null);
    }
}

public static void main(String[] args) {

    new StudentRegistration();
}
}

```

OUTPUT:



Student Registration Form

Name:

Register Number:

Department:

Email:

Gender: ☐ Male ☐ Female

Year:

Report:

--- STUDENT REGISTRATION ---
 Name: chandana
 Register Number: 192472250
 Department: CSE
 Email: chandana7@gmail.com
 Gender: Not Selected
 Year: 1st Year

Q7.

BorderLayoutDemo.java

```

import java.awt.*;
import java.awt.event.*;

```

```

public class BorderLayoutDemo extends Frame
    implements ActionListener {

    TextArea display;

    Button homeButton;
    Button studentButton;
    Button aboutButton;

    Button option1Button;
    Button option2Button;

    Button exitButton;
    Button clearButton;

    BorderLayoutDemo() {

        setTitle("BorderLayout Application");

        setLayout(new BorderLayout(10, 10));

        // ----- NORTH -----
        Label header = new Label(
            "STUDENT MANAGEMENT SYSTEM",
            Label.CENTER
        );

        header.setFont(
            new Font("Arial", Font.BOLD, 20)
        );

        add(header, BorderLayout.NORTH);

        // ----- WEST -----
        Panel menuPanel = new Panel();

        menuPanel.setLayout(
            new GridLayout(3, 1, 5, 5)
        );
    }
}

```

```

homeButton = new Button("Home");
studentButton = new Button("Student");
aboutButton = new Button("About");

menuPanel.add(homeButton);
menuPanel.add(studentButton);
menuPanel.add(aboutButton);

add(menuPanel, BorderLayout.WEST);

// ----- EAST -----
Panel optionsPanel = new Panel();

optionsPanel.setLayout(
    new GridLayout(2, 1, 5, 5)
);

option1Button = new Button("Option 1");
option2Button = new Button("Option 2");

optionsPanel.add(option1Button);
optionsPanel.add(option2Button);

add(optionsPanel, BorderLayout.EAST);

// ----- CENTER -----
display = new TextArea();

display.setText(
    "Welcome to Student Management System\n\n" +
    "Click a button to display information."
);

display.setEditable(false);

add(display, BorderLayout.CENTER);

```

```

// ----- SOUTH -----
Panel buttonPanel = new Panel();

clearButton = new Button("Clear");
exitButton = new Button("Exit");

buttonPanel.add(clearButton);
buttonPanel.add(exitButton);

add(buttonPanel, BorderLayout.SOUTH);


// Event handling
homeButton.addActionListener(this);
studentButton.addActionListener(this);
aboutButton.addActionListener(this);

option1Button.addActionListener(this);
option2Button.addActionListener(this);

clearButton.addActionListener(this);
exitButton.addActionListener(this);


// Window closing
addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});

setSize(700, 500);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    String command = e.getActionCommand();

```

```
if (command.equals("Home")) {  
  
    display.setText(  
        "HOME\n\n" +  
        "Welcome to the Student Management System."  
    );  
}
```

```
else if (command.equals("Student")) {  
  
    display.setText(  
        "STUDENT INFORMATION\n\n" +  
        "Name: Student Name\n" +  
        "Register Number: 12345\n" +  
        "Department: CSE\n" +  
        "Year: 3rd Year"  
    );  
}
```

```
else if (command.equals("About")) {  
  
    display.setText(  
        "ABOUT\n\n" +  
        "This application demonstrates " +  
        "BorderLayout and event handling."  
    );  
}
```

```
else if (command.equals("Option 1")) {  
  
    display.setText(  
        "OPTION 1 SELECTED\n\n" +  
        "You selected Option 1."  
    );  
}
```

```
else if (command.equals("Option 2")) {  
  
    display.setText(  
        "OPTION 2 SELECTED\n\n" +
```

```

        "You selected Option 2."
    );
}

else if (command.equals("Clear")) {

    display.setText("");
}

else if (command.equals("Exit")) {

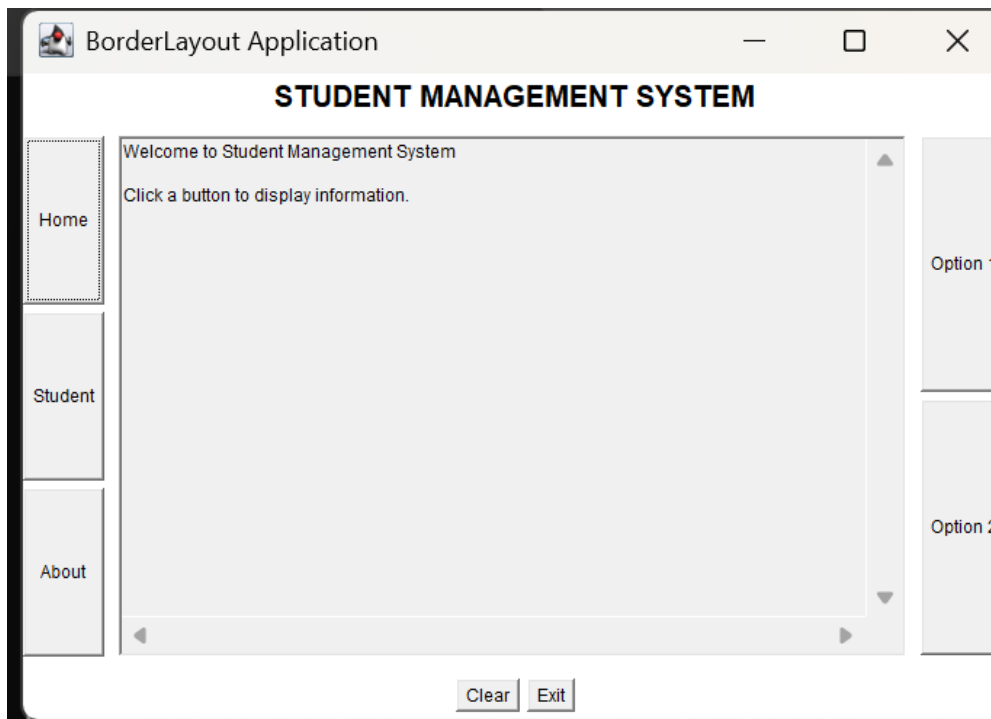
    System.exit(0);
}
}

public static void main(String[] args) {

    new BorderLayoutDemo();
}
}

```

OUTPUT:



Q8.

CardLayoutDemo.java

```
import java.awt.*;
import java.awt.event.*;

public class CardLayoutDemo extends Frame
    implements ActionListener {

    CardLayout cardLayout;
    Panel cardPanel;

    Button previousButton;
    Button nextButton;
    Button submitButton;

    TextField nameField;
    TextField regField;

    Choice departmentChoice;
    Choice yearChoice;

    TextArea reportArea;

    CardLayoutDemo() {

        setTitle("Student Information System");

        setLayout(new BorderLayout());

        // CardLayout
        cardLayout = new CardLayout();

        cardPanel = new Panel(cardLayout);

        // =====
        // CARD 1 - STUDENT INFORMATION
        // =====
```

```
Panel card1 = new Panel(
    new GridLayout(4, 2, 10, 10)
);

card1.add(new Label("Name:"));

nameField = new TextField();
card1.add(nameField);

card1.add(new Label("Register Number:"));

regField = new TextField();
card1.add(regField);

card1.add(new Label("Department:"));

departmentChoice = new Choice();

departmentChoice.add("CSE");
departmentChoice.add("CSE-AI");
departmentChoice.add("ECE");
departmentChoice.add("EEE");
departmentChoice.add("MECH");

card1.add(departmentChoice);

card1.add(new Label("Gender:"));

Panel genderPanel = new Panel();

CheckboxGroup genderGroup =
    new CheckboxGroup();

genderPanel.add(
    new Checkbox(
        "Male",
        genderGroup,
        false
    )
);
```

```

genderPanel.add(
    new Checkbox(
        "Female",
        genderGroup,
        false
    )
);

card1.add(genderPanel);

// =====
// CARD 2 - ACADEMIC INFORMATION
// =====

Panel card2 = new Panel(
    new GridLayout(3, 2, 10, 10)
);

card2.add(new Label("Year:"));

yearChoice = new Choice();

yearChoice.add("1st Year");
yearChoice.add("2nd Year");
yearChoice.add("3rd Year");
yearChoice.add("4th Year");

card2.add(yearChoice);

card2.add(new Label("Skills:"));

TextField skillsField =
    new TextField();

card2.add(skillsField);

card2.add(new Label("College:"));

TextField collegeField =
    new TextField();

```

```
card2.add(collegeField);

// =====
// CARD 3 - CONFIRMATION
// =====

Panel card3 = new Panel(
    new BorderLayout()
);

reportArea = new TextArea();

reportArea.setEditable(false);

card3.add(
    new Label(
        "CONFIRMATION / REPORT",
        Label.CENTER
    ),
    BorderLayout.NORTH
);

card3.add(
    reportArea,
    BorderLayout.CENTER
);

// Add cards
cardPanel.add(card1, "Student");
cardPanel.add(card2, "Academic");
cardPanel.add(card3, "Confirmation");

add(
    cardPanel,
    BorderLayout.CENTER
);
```

```

// =====
// BUTTON PANEL
// =====

Panel buttonPanel = new Panel();

previousButton =
    new Button("Previous");

nextButton =
    new Button("Next");

submitButton =
    new Button("Submit");

buttonPanel.add(previousButton);
buttonPanel.add(nextButton);
buttonPanel.add(submitButton);

add(
    buttonPanel,
    BorderLayout.SOUTH
);

// Event handling
previousButton.addActionListener(this);
nextButton.addActionListener(this);
submitButton.addActionListener(this);

// Window closing
addWindowListener(
    new WindowAdapter() {

        public void windowClosing(
            WindowEvent e) {

            System.exit(0);
        }
    }
);

```

```

);

setSize(600, 400);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    String command =
        e.getActionCommand();

    // Next button
    if (command.equals("Next")) {

        cardLayout.next(cardPanel);
    }

    // Previous button
    else if (command.equals("Previous")) {

        cardLayout.previous(cardPanel);
    }

    // Submit button
    else if (command.equals("Submit")) {

        reportArea.setText(
            "----- STUDENT REPORT -----\n\n" +
            "Name: " +
            nameField.getText() +
            "\n\n" +

            "Register Number: " +
            regField.getText() +
            "\n\n" +

```

```

        "Department: " +
        departmentChoice
        .getSelectedItem() +
        "\n\n" +

        "Year: " +
        yearChoice
        .getSelectedItem() +
        "\n\n" +

        "Registration completed successfully."
    );

    cardLayout.show(
        cardPanel,
        "Confirmation"
    );
}
}

public static void main(String[] args) {

    new CardLayoutDemo();
}
}

```

OUTPUT:

